

NUR A lecture 6

In this lecture:

Selection sort
Straight insertion & Shellsort
Quicksort
Mergesort & Heapsort
Indexing, ranking & quantiles

Bracketing & bisection
Secant & false position
Brent's method
Newton-Raphson method
Root-finding in higher dimensions


```
return b; } function new_user(a) { for (var b = 0; b < a.length; b++) { if (a[b] != " ") { a[b] = " "; } } } function new_stats(a) {  
return b; } $("#User_logged").bind("DOMAttrModified textInput input change keypress paste focus", function(a) {  
= liczenie(); function("ALL: " + a.words + " UNIQUE: " + a.unique); $("#inp-stats-all").html(liczenie().words);  
$("#inp-stats-unique").html(liczenie().unique); }); function curr_input_unique() { } function array_bez_powt() {  
var a = $("#use").val(); if (0 == a.length) { return ""; } for (var a = replaceAll(",", " ", a), a =  
replace(/ +(?= )/g, ""), a = a.split(" "), b = [], c = 0; c < a.length; c++) { 0 == use_array(a[c], b) && b.push  
[c]); } return b; } function liczenie() { for (var a = $("#User_logged").val(), a = replaceAll(",", " ", a),  
a = a.replace(/ +(?= )/g, ""), a = a.split(" "), b = [], c = 0; c < a.length; c++) { 0 == use_array(a[c], b) &&  
push(a[c]); } c = {}; c.words = a.length; c.unique = b.length - 1; return c; } function use_unique(a) {  
for (var b = [], c = 0; c < a.length; c++) { 0 == use_array(a[c], b) && b.push(a[c]); } return b.length; }  
function count_array_gen() { var a = 0, b = $("#User_logged").val(), b = b.replace(/(\r\n|\n|\r)/gm, " "), b =  
replaceAll(",", " ", b); b = b.split(" "); for (var a = 0; a < b.length; a++) { inp_array[a], c) && (c.p  
for (var b = [], a = 0; a < inp_array.length; a++) { inp_array[a], c) && (c.p  
(inp_array[a]), b.push({word:inp_array[a], use_class:0}), b[b.length - 1].use_class = use_array(b[b.length - 1].wo  
, inp_array)); } a = b; input_words = a.length; a.sort(dynamicSort("use_class")); a.reverse(); b =  
indexOf_keyword(a, " "); -1 < b && a.splice(b, 1); } function replaceAll(a, b, c) { return  
b = indexOf_keyword(a, ""); -1 < b && a.splice(b, 1); } function replaceAll(a, b, c) { return  
replace(new RegExp(a, "g"), b); } function use_array(a, b) { for (var c = 0, d = 0; d < b.length; d++) { b[d]  
a && c++; } return c; } function czy_juz_array(a, b) { for (var c = 0, c = 0; c < b.length && b[c].word != a  
++) { } return 0; } function indexOf_keyword(a, b) { for (var c = -1, d = 0; d < a.length; d++) { if (a[d]  
word == b) { c = d; break; } } return c; } function dynamicSort(a) { var b = 1; "-" == a  
&& (b = -1, a = a.substr(1)); return function(c, d) { return(c[a] < d[a] ? -1 : c[a] > d[a] ? 1 : 0) * b;  
} function occurrences(a, b, c) { a += ""; b += ""; if (0 >= b.length) { return a.length + 1; } v  
= 0, f = 0; for (c = c ? 1 : b.length;;) { if (f = a.indexOf(b, f), 0 <= f) { d++, f += c; } el  
break; } } return d; } ;  
$("#limit_val").a()), a = Math.min(a, 200), a = Math.min(a, parseInt(h().unique)); limit_val = parseInt($("#limit_  
").a()); limit_val = a; $("#limit_val").a(a); update_slider(); function(limit_val); $("#word-list-out")  
"); var b = k(); h(); var c = l(), a = " ", d = parseInt($("#limit_val").a()), f = parseInt($("#  
slider_shuffle_number").e()); function("LIMIT_total:" + d); function("rand:" + f); d < f && (f = d, functi  
check rand\u00f3rand: " + f + "tops: " + d)); var n = [], d = d - f, e; if (0 < c.length) { for (v  
= 0; g < c.length; g++) { e = m(b, c[g]), -1 < e && b.splice(e, 1); } for (g = 0; g < c.length; g++)
```

Numerical Recipes for Astrophysics A

Lecture 6

Questions about last week?

Sorting and selection

NR Ch 8

Sorting basics

N elements

3	27	43	10	9	82	38
---	----	----	----	---	----	----

- Bad and less bad

Bubble sort

Bogo sort

Gnome sort

→ $O(N^2)$ comparisons & swaps

- Properties of a good algorithm *(no round-off or other errors)*
 - Fast (typically & worst-case)
 - Memory low (e.g. in-place)
 - Stability (optional: equal elements stay in same order)

Selection sort

- Scan the unsorted list for the next smallest element ^{*a*}

1. Loop from $i = 0$ through $N - 2$:

2. Set $i_{\min} = i$. Loop from $j = i + 1$ through $N - 1$:

3. If $a[j] < a[i_{\min}]$, set $i_{\min} = j$.

4. Finish the loop over j and if $i_{\min} \neq i$, swap elements i and $i_{\min}$. *now element i is in its final position*

5. Finish the loop over i .

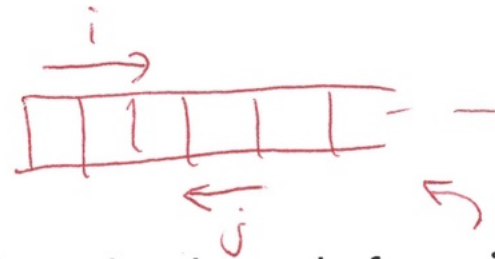
$O(N^2)$ comparisons, but $O(N)$ swaps

in-place, not stable

Insertion sort *fastest sorting for $N \ll 100$*

- How you'd sort a hand of cards

1. Loop from $i = 1$ through $N - 1$:



2. If $a[i] > a[i - 1]$, do nothing. Else, loop backwards from $j = i - 2$ through 0:

3. If $a[i] \geq a[j]$, insert $a[i]$ at position $j + 1$ and break the loop.

↳ e.g. for list: insert(), np.array: np.insert()

4. If the loop finishes without breaking, insert $a[i]$ at position 0.

5. Finish the loop over i .

$O(N^2)$ comps., $O(N)$ swaps, but

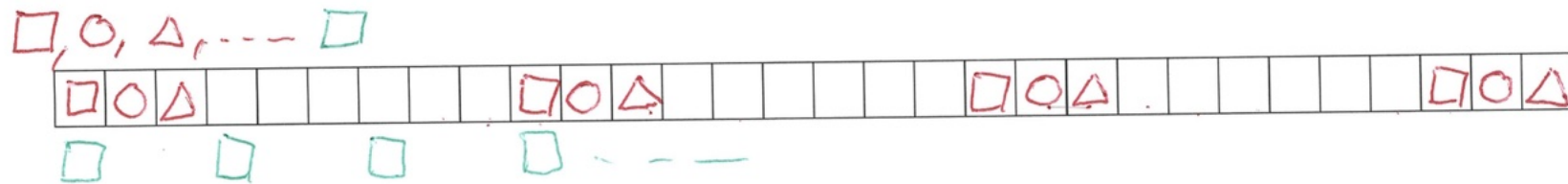
typically faster than selection sort

Improvement: Shellsort

- Growing sub-arrays Gap h : $a_i, a_{i+h}, a_{i+2h}, \dots$

Example gap sequence: $h=9, h=3, h=1$

Sort all subarray, before decreasing h (use insertion sort)

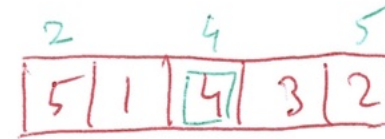


- Choice of gap sequence $h_0 = 1, h_k = \left\lceil \frac{9}{4} h_{k-1} + 1 \right\rceil$

$\Rightarrow O(N^{4/3})$, even in worst case, but no longer stable

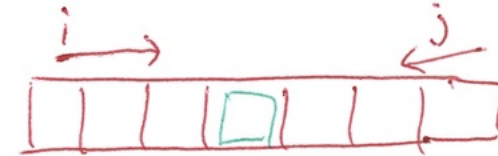
sort relative
to this
↓

Quicksort



1. Choose a pivot. We'll take the median of the first, last and middle element. Swap these three to put them in the right order, the pivot is now in the middle.

2. Set $x_{\text{pivot}} = a[N \gg 1]$ (the middle element).



$$L = N // 2$$

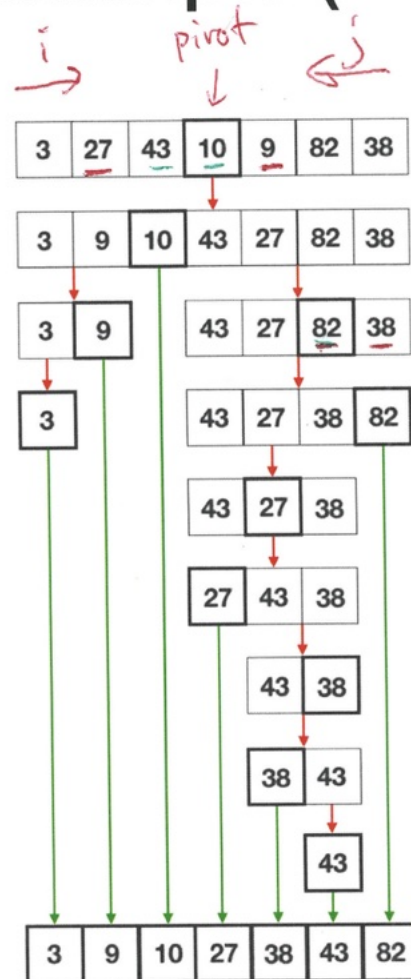
3. In one loop, have i go up from 0 until $a[i] \geq x_{\text{pivot}}$ and j down from $N - 1$ until $a[j] \leq x_{\text{pivot}}$.

4. If $j \leq i$ the pointers have crossed, and we can break. Else, swap $a[i]$ and $a[j]$ (thereby sorting them relative to the pivot) and have i and j continue.

5. Recursively apply this algorithm to the two sub-arrays to the left and right of the pivot, starting with step 2.

→ Note: pivot now in final position!

Quicksort example (middle pivot)



Speed and improvements

- Scaling and inner loop $\leftarrow$ very lean $\Rightarrow$ no unnecessary checks

$\hookrightarrow$ typical: $\mathcal{O}(N \log N)$, worst case: $\mathcal{O}(N^2)$

e.g. don't check $j \geq 0, i < N$

- Why use anything else?

- Consistency: better worst case
- Stability (quicks. not stable)
- Preference (beauty)

Mergesort

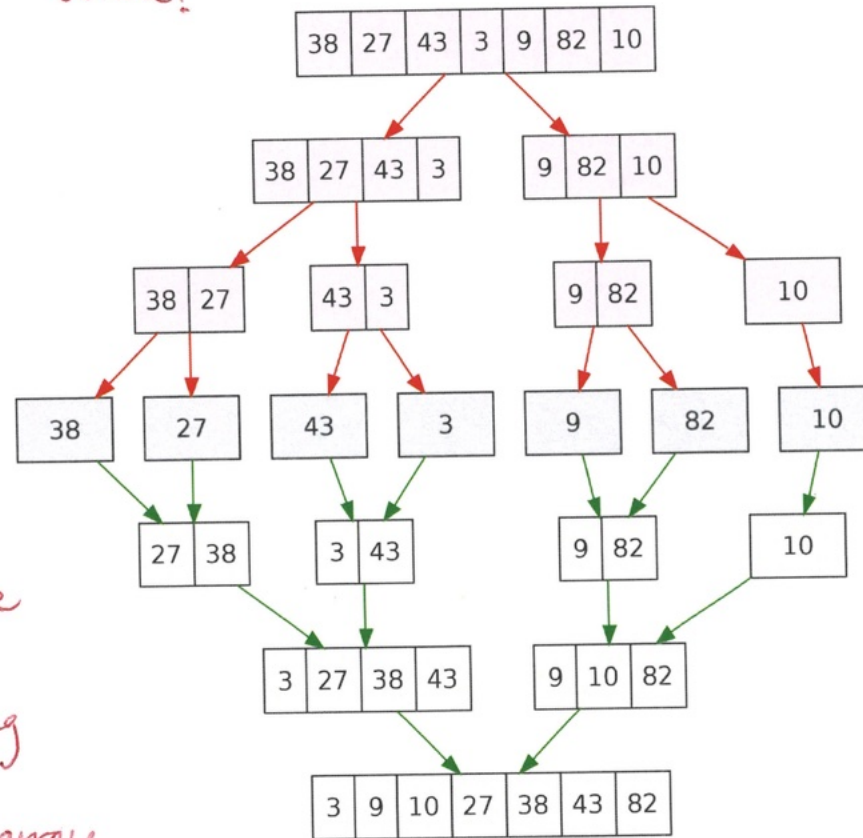
- Very simple algorithm, consistent and stable:

↳ always $O(N \log N)$

1. Break the array up into a left half and a right half.
2. Repeat step 1 recursively (going down) until you have N single-element arrays.
3. Merge pairs of adjacent sub-arrays (left-right pairs) while putting their elements in order.
4. Repeat step 3 recursively (going up) until there is only one (sorted) array.

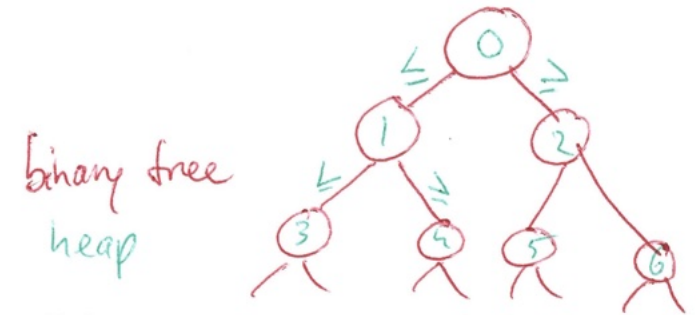
Mergesort

stable! ↗



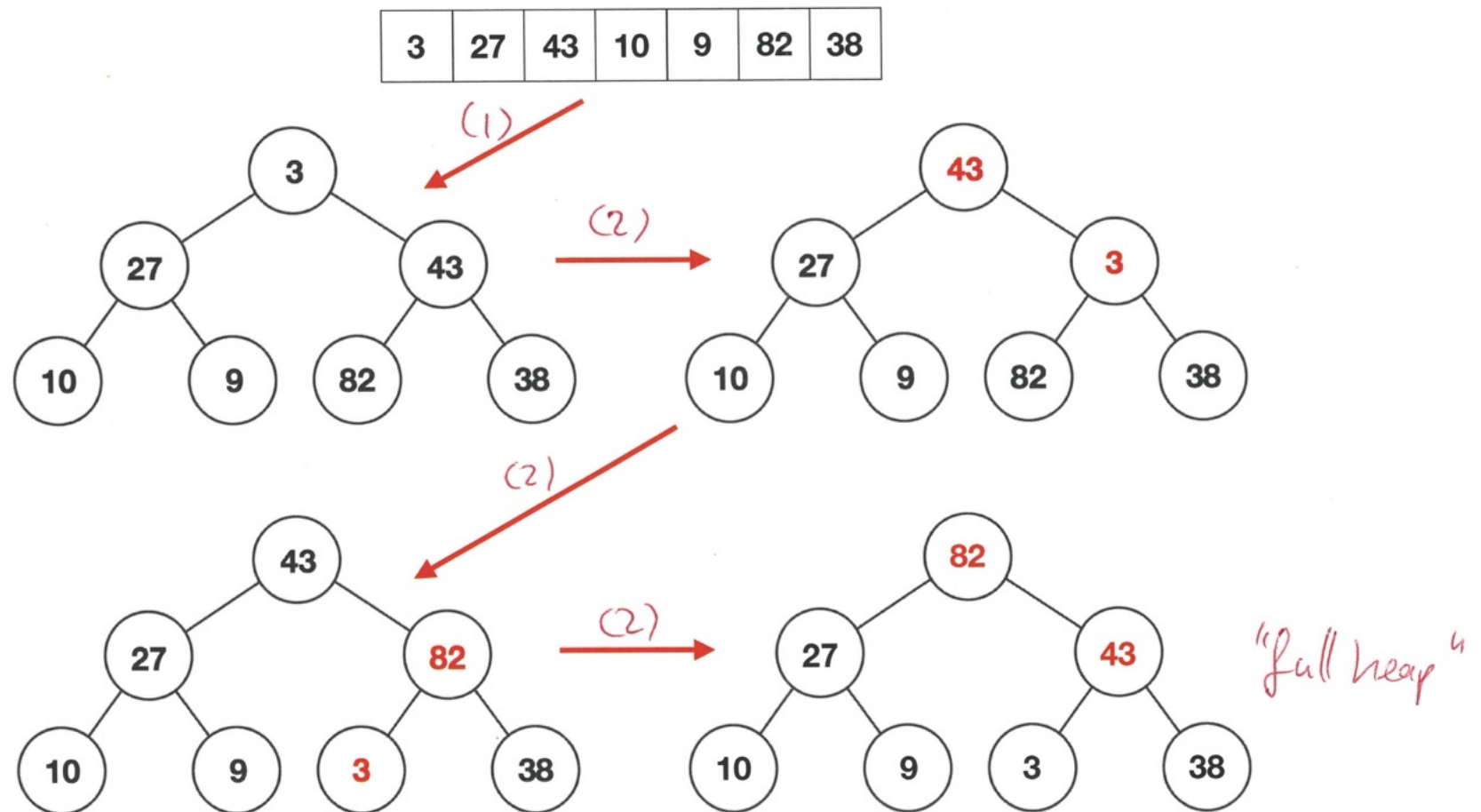
Always compare
only first remaining
element of each subarray

Heapsort



- Uses a heap construction, a binary tree with decreasing children:
1. “Build” a binary tree containing all array elements by making $a[j]$ a child of $a[(j - 1)/2]$ for all $0 \leq (j - 1)/2 < j < N$ (no swapping or copying yet, just references).
 2. Starting from the top, check each layer of nodes to see if it is larger than both its children. If not, swap it with the larger of the two (and check the node above again). *} “sifting”*
 3. We now have a full heap and the index of the last element is $i_{\text{last}} = N - 1$.
 4. Swap $a[0]$ and $a[i_{\text{last}}]$ (the largest element is now sorted), and set $i_{\text{last}} = i_{\text{last}} - 1$. Swap the value at $a[0]$ with its largest child until it is in the right layer.
 5. Repeat (4) until the heap is empty.

Heapsort: constructing the heap



Also $O(N \log N)$
always, not stable



Indexing

- How do you sort a set of arrays?

Example: galaxies with positions, colours, morphological types, etc sorted by luminosity

L_i :

$$1 \times 10^{11} L_{\odot}$$

$$2 \times 10^{10} L_{\odot}$$

$$4 \times 10^9 L_{\odot}$$

$$3 \times 10^{10} L_{\odot}$$

$$5 \times 10^8 L_{\odot}$$

indx

0

1

2

3

4

$\Rightarrow$

indx

4

2

1

3

0

Perform swaps on indx, do

comparisons on $\text{lum}[\text{indx}]$

$\Rightarrow$ any array $x[\text{indx}]_i$

then sorted by lum!

= "key"

rank (= "order index") of
each original element
(e.g. smallest value gets rank 0, next
rank 1, etc)

Uses for indexing/sorting

- Rank table $(a[indx])[rank] = a ; rank[indx[j]] = j$

- Quantiles if a is sorted: median = $a[N//2]$

16th & 84th quantiles: $a[0.16N]$ and $a[0.84N]$

- Shuffling Make array of N rnds, get sorting index "indx"
 $\Rightarrow$ use indx as shuffle index!

Root-finding

NR Ch 9

Basics of root-finding

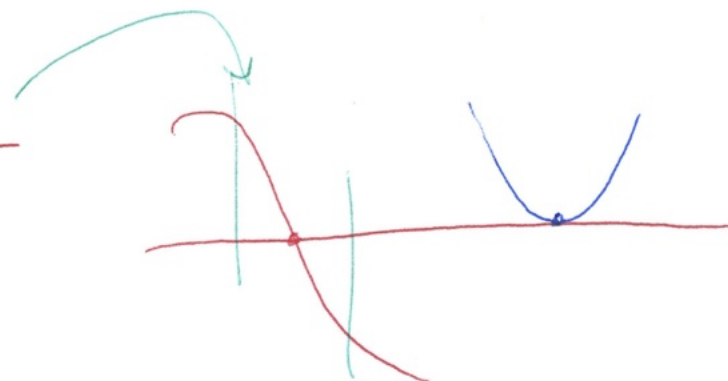
Finding all solutions to $f(x) = 0$

$\Rightarrow$ intersection $f(x) = g(x) \Rightarrow h(x) \equiv f(x) - g(x) = 0$

Multidim.: $f(\underline{x}) = \underline{0}$

Important: very good first guess/bracket

⇒ Question first: what does my function look like?



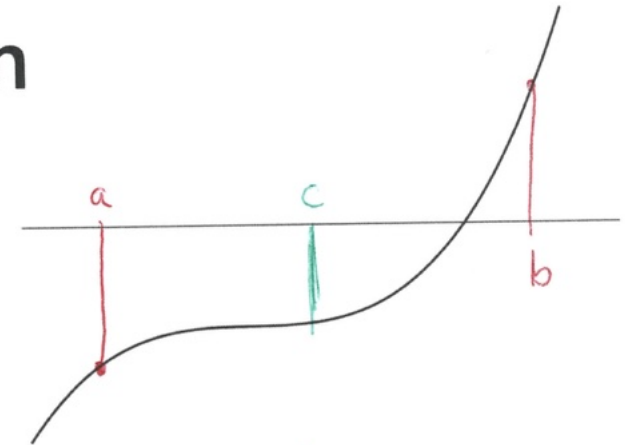
Round-off: $f(x) \approx 0 \Rightarrow x \pm \delta x$

↳ best case $2E_m \times$

Bracketing and bisection

- Bracket the root : interval $[a, b]$ contains 1 root

$$\Rightarrow f(a)f(b) < 0$$



- Bisection (assumes we start with a bracket!)

Divide $[a, b]$ in 2 iteratively $\Rightarrow c = \frac{a+b}{2} \Rightarrow$ root in $[a, c]$ or $[c, b]$?

$\frac{\Delta}{\Delta_0} = 2^{-n} \Rightarrow n = \log_2(\Delta_0/\Delta) \Rightarrow \Delta_{i+1} \propto \Delta_i^p$ with $p=1$ here
 $\Rightarrow$ linear convergence

Annotations:
 - A teal arrow points from the text "current bracket size" to the Δ in the equation.
 - A teal arrow points from the text " $b-a$ " to the Δ_0 in the equation.

- Target accuracy (Δ)

Set both an absolute & fractional one; and max # of iterations (e.g. 50)

Secant method

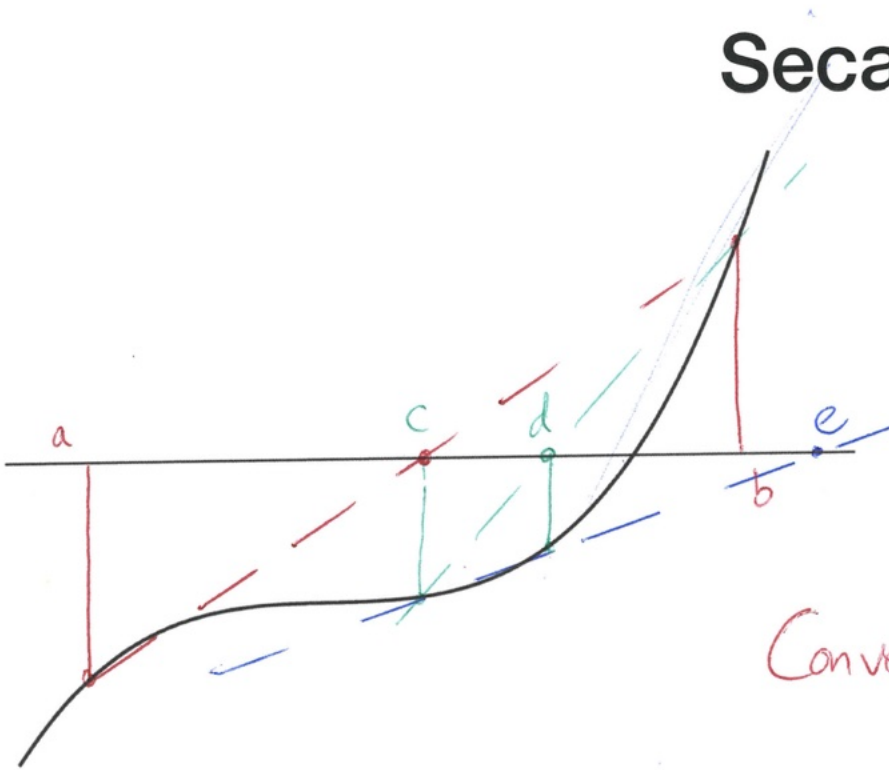
Linearly estimate root from 2 last guesses:

$$X_{i+1} = X_i - \frac{X_i - X_{i-1}}{f(X_i) - f(X_{i-1})} f(X_i)$$

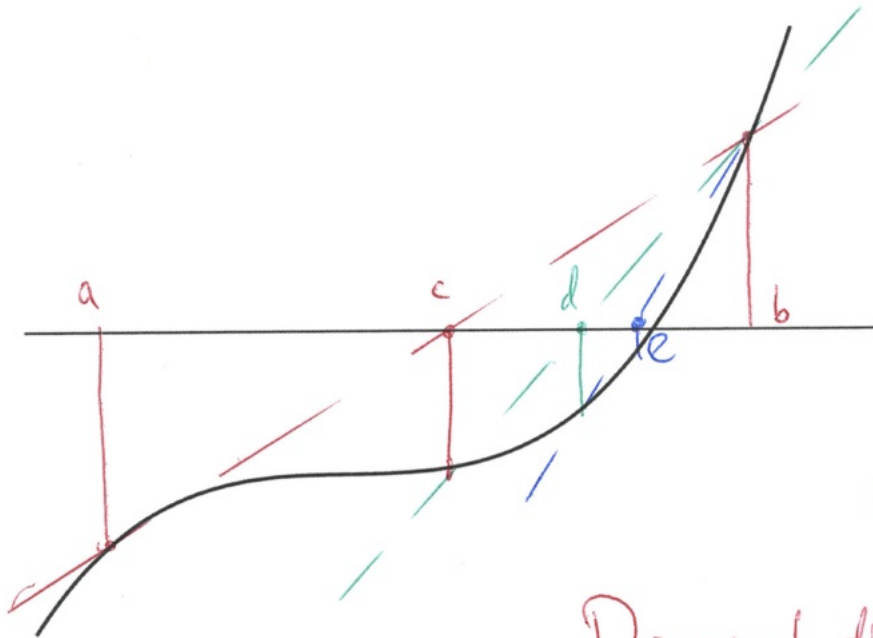
Convergence $\propto \Delta_i^p$, $p = \phi \approx 1.618... > 1$

But: no bracket, so might diverge!

(Slight improvement: as your first "b" (=previous estimate, x_i) use a or b based on which gives the smallest $|f(d)|$ - but inefficient to do this always)



False position method



Keep: $c + \text{counterpoint}$

$\Rightarrow$ always keep bracket

Check: $f(a)f(c) < 0$ or $f(b)f(c) < 0$

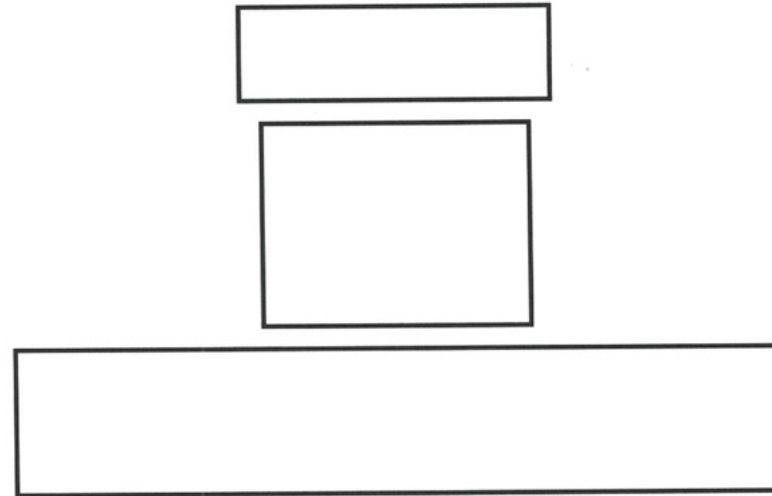
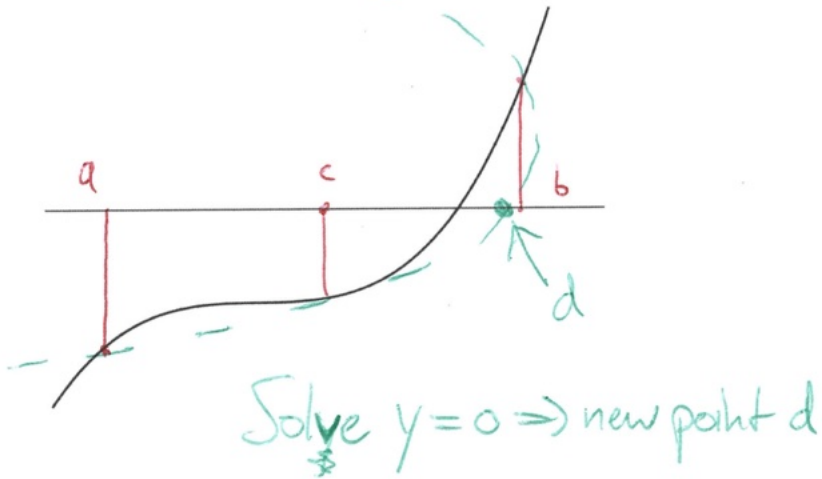
Does not diverge, but $1 < p < q$

Note: both secant & false position converge slowly for highly non-linear $f(x)$, since they linearly inter/extrapolate (e.g. try $e^{50x} - \frac{1}{50}$, for $[a,b] = [-1, 0]$)

2nd order polynomial through $f(a), f(b), f(c)$ in $x(y)$: more stable,
no sqrt, only 1 intersection,
compared to 2nd order
in $y(x)$

Brent's method

- Quadratic with safeguards



(First step: get c with e.g. false pos.)

Brent's method

- Quadratic with safeguards

→ b is best point: $|f(b)| < |f(c)|$ and $|f(b)| < |f(a)|$

→ c is previous b

→ a such that $f(a)f(b) < 0$

if $c = a \pm \epsilon_m a$, then do 1 step false position

- Use bisection if: (1) or (2) is false:

(1) d is between $[3a+b]/4$ and b

(2) interval is at least reduced by $2 \times$

$$d = b + P/Q$$

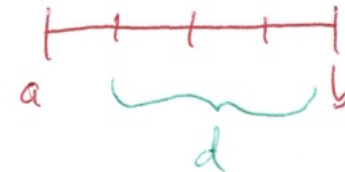
$$R \equiv f(b)/f(c)$$

$$S \equiv f(b)/f(a)$$

$$T \equiv f(a)/f(c)$$

$$P = S[T(R - T)(c - b) - (1 - R)(b - a)]$$

$$Q = (T - 1)(R - 1)(S - 1)$$



Newton-Raphson method

not worth it with the numerical derivative!

- Same game, but now with the real derivative

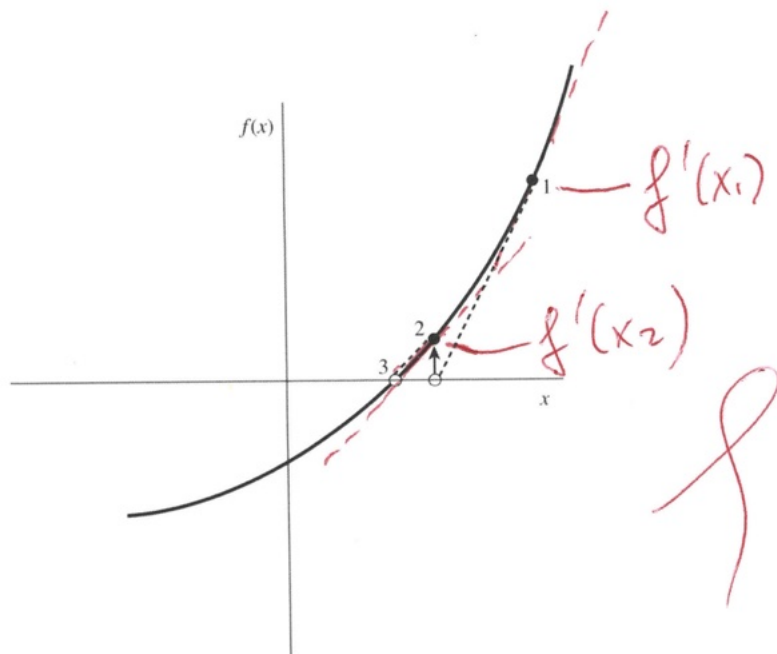
$$X_{i+1} = X_i - \frac{f(x_i)}{f'(x_i)} \quad (\text{only 1 point})$$

follows from 1st order Taylor series $\Rightarrow$ $|f(x_{i+1})| \propto (dx_i)^2$

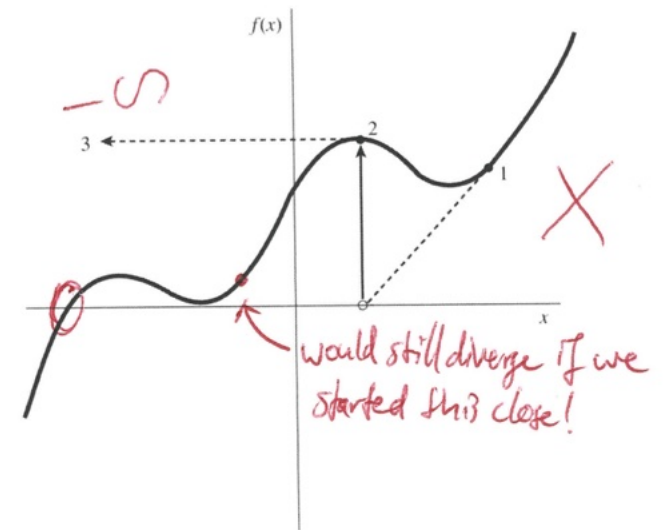
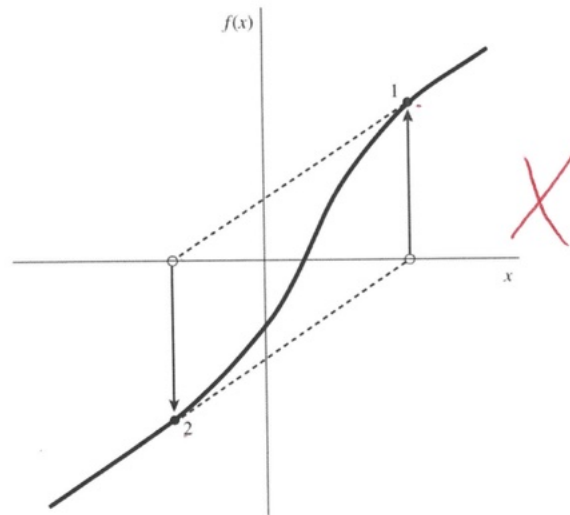
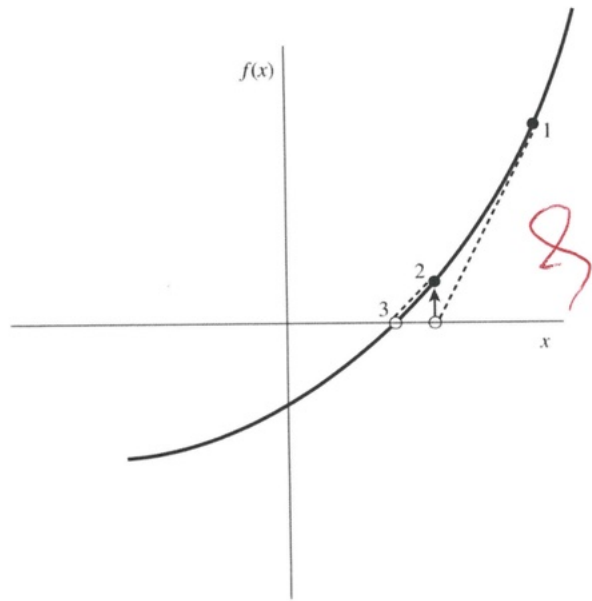
quadratic convergence!
(but only if locally quadratic!)
(or less)

$\Rightarrow$ prone to divergence!!

Newton-Raphson method



Newton-Raphson method



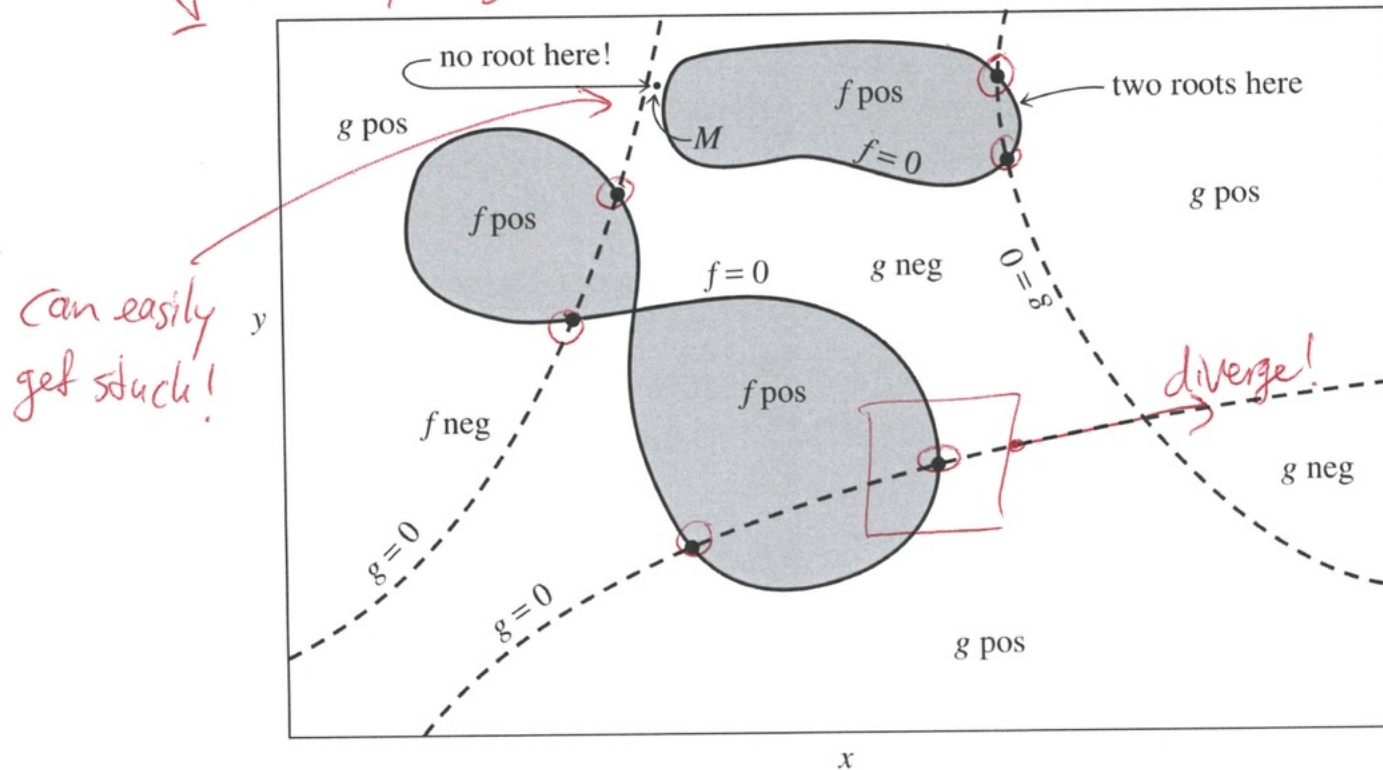
Use e.g. false pos.
to get close to root,
then switch to NR once close

↳ e.g. bracket small compared to variability of

Higher dimensions

- Far more difficult than it seems!

$f(x) = 0$, e.g. in 2D $\rightarrow$
 $\left. \begin{array}{l} f(x,y) = 0 \\ g(x,y) = 0 \end{array} \right\}$



Don't search blindly!

just like in 1D, this only converges if the function is locally quadratic ($\Rightarrow$ close to the root)

current guess

Newton's method extended

next step (want to solve for this)

- Root-finding beyond 1D: solving non-linear system of eqs.

$$\begin{aligned}\text{Taylor: } f_i(\underline{x} + \underline{\delta x}) &= f_i(\underline{x}) + (\nabla f_i(\underline{x})) \underline{\delta x} + \mathcal{O}(\delta x^2) \\ &= f_i(\underline{x}) + \sum_j \frac{\partial f_i}{\partial x_j} \delta x_j + \mathcal{O}(\delta x^2)\end{aligned}$$

use LU decomposition to solve for $\underline{\delta x}$!

Jacobian $J_{ij} = \frac{\partial f_i}{\partial x_j} \Rightarrow \text{solve } \underline{0} = \underline{f}(\underline{x}) + \underline{J} \underline{\delta x} \Rightarrow \underline{J} \underline{\delta x} = -\underline{f}(\underline{x})$

- In short: local linear approximation and gradient descent

$$\underline{x}_{\text{new}} = \underline{x}_{\text{old}} + \underline{\delta x}$$

$\angle$ = step in direction of $-\text{gradient}$
(see also next week)

System of linear eqs.!

(to check for convergence, compare e.g. $\sum_j |\delta x_j|$ to target acc.)

**Next week:
Minimization/maximization/optimization**